

7장 트랜스포머 (1)

1 Transformer

1. 입력 임베딩과 위치 인코딩
2. 특수 토큰
3. 트랜스포머 인코더
4. 트랜스포머 디코더
5. 모델 실습

2 GPT

1. GPT-1
2. GPT-2
3. GPT-3
4. GPT-3.5
5. GPT-4
6. 모델 실습

• 트랜스포머(Transformer)

- 2017년 코넬대학의 아시시 바스와니(Ashish Vaswani) 등의 연구 그룹이 발표한 [Attention is All You Need] 논문을 통해 소개된 신경망 아키텍처
- 주요 기능 : 병렬로 입력 시퀀스를 처리하는 기능 ↔ 기존 순환 신경망 (순차적 방식)
 - 긴 시퀀스의 경우 트랜스포머 모델이 순환 신경망 모델보다 훨씬 더 빠르고 효율적으로 처리함
- ∵ 순차 처리나 반복 연결에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 셀프 어텐션 기반
 - 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계를 직접 모델링 가능
- 대용량 데이터 세트 학습 에서 매우 효율적이며 데이터의 양이 많은 기계 번역과 같은 작업에 적합
- 언어 모델링 및 텍스트 분류와 같은 작업에서 매우 효과적, 광범위한 자연어 처리 작업에서 높은 효율
- 특히 기계번역, 언어모델링, 텍스트요약과 같은 장기적인 종속성을 포함하는 작업에 주로 사용됐으며, 자연어 처리 분야에서 널리 사용되고 영향력이 큰 모델

• 오토인코딩 / 자기 회귀 or 두 개의 조합으로 학습

- 오토 인코딩 방식 : 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업 수행
- 인코더(Encoder) : 예측되는 토큰의 양 옆에 있는 토큰들을 참조 → 양방향구조
- 트랜스포머 구조



그림 7.1 트랜스포머 구조

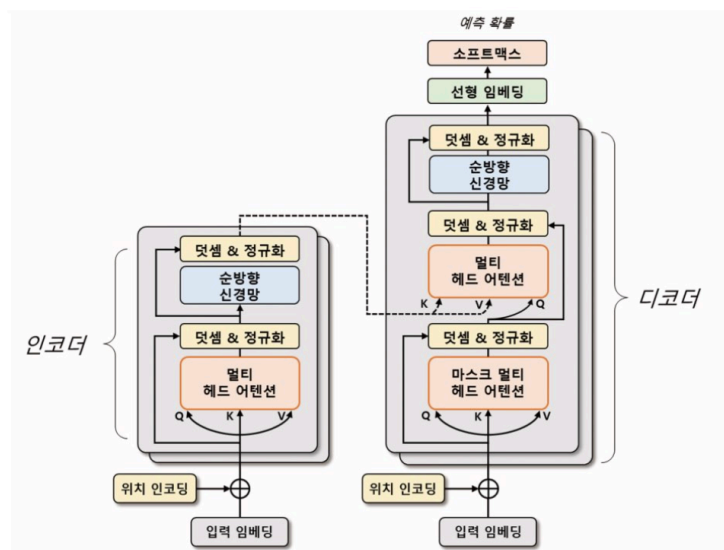
- A : '트랜스포머 모델은 성능이', '다양한 구조로 활용된다' 입력으로, '높고'를 출력으로 학습하는 양방향 구조
 - but 자기 회귀 방식 → 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업수행
- B : '트랜스포머 모델은 성능이'를 입력, '높고'를 출력으로 학습하는 단방향 구조
 - 디코더 : 예측되는 토큰의 왼쪽에 있는 토큰들만 참조 → 단방향 구조
- 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

1 Transformer

- 딥러닝 모델 중 하나로, 기계번역, 챗봇, 음성인식 등 다양한 자연어 처리 분야에서 많은 성과를 내는 모델
- 순환 신경망이나 합성곱 신경망과 달리 어텐션 메커니즘 만을 사용하여 시퀀스 임베딩을 표현

- 어텐션 메커니즘
 - 인코더와 디코더 간의 상호 작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력 생성
 - 인코더 : 입력 시퀀스를 임베딩하여 고차원 벡터로 변환
 - 디코더 : 인코더의 출력을 입력으로 받아 출력 시퀀스 생성
- ⇒ 인코더와 디코더 단어 사이의 상관 관계를 계산하여 중요한 정보에 집중
- ⇒ 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어로 1 관련이 있는지를 파악하여 번역이나 요약문 생성과 관련된 작업 수행 가능
- 기존 순환 신경망 기반 모델보다 학습 속도 ↑ , 병렬 처리 가능 → 대규모 데이터 세트에서 높은 성능 , 임베딩 과정에서 문장의 전체 정보 고려하기 → 문장의 길이가 길어지더라도 성능 유지 가능
- 트랜스포머 모델 구조



- 인코더, 디코더 : 두 부분으로 구성, 각각 N개의 트랜스포머 블록으로 구성
 - 트랜스포머 블록 = 멀티 헤드 어텐션 + 순방향 신경망
 - 멀티 헤드 어텐션 : 입력시퀀스에서 쿼리(Query), 키(Key),값(Value) 벡터 정의 → 입력 시퀀스들의 관계를 셀프 어텐션 하는 벡터 표현 방법
= 쿼리와 각 키의 유사도 계산 → 해당 유사도를 가중치로 사용 → 값 벡터를 합산
→ 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체

- 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신
- 순방향 신경망 : ↑ 과정에서 산출된 임베딩 벡터를 더욱 고도화 하기 위해 사용
 - 여러 개의 선형계층으로 구성
 - 입력 벡터에 가중치 곱하고, 편향 더하며, 활성화 함수 적용 (앞선 순방향 신경망의 구조와 동일)
 - 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신
- 입력 시퀀스 데이터 → 소스(Source) / 타겟(Target) 데이터로 나눠 처리
 - ex. 영어 → 한글 번역 : 생성하는 언어인 한글 = 타겟데이터 정의, 참조하는 언어인 영어 = 소스데이터 정의
- 인코더: 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현 → 트랜스 포머 블록의 출력벡터 생성
 - 출력 벡터 : 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성
- 디코더 : 인코더와 유사하게 트랜스포머 블록으로 구성 but, 마스크 멀티 헤드 어텐션사용 → 타겟 시퀀스 데이터를 순차적으로 생성
 - 이때 디코더 입력 시퀀스들의 관계를 고도화 하기 위해 인코더의 출력 벡터 정보 참조
- 최종적으로 생성된 디코더 출력 벡터 → 선형 임베딩으로 재표현 → 이미지나 자연어모델에 활용
 - ⇒ 챗봇, 기계번역, 음성인식 등 다양한 자연어 처리 작업에서 좋은 성능 보임

1. 입력 임베딩과 위치 인코딩

- 트랜스 포머 모델에서 입력시퀀스의 각 단어 → 임베딩 처리 → 벡터 형태로 변환
- 입력 시퀀스 병렬 구조 처리 → 단어의 순서 정보 제공 X ($\leftrightarrow$ 순환 신경망)
 - ∴ 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보 모델에 반영 필요
 - ⇒ 이를 위해 위치 인코딩 방식 사용
- 위치 인코딩
 - 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
 - 각단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보 반영

- sin 함수, cos 함수 사용해 생성 → 임베딩 벡터와 위치 정보 결합된 최종 입력 벡터 생성
 - 위치 인코딩 벡터 추가 → 모델은 단어의 순서정보 학습 가능
- 각 토큰의 위치를 각도로 표현 → sin 함수, cos 함수 사용해 위치 인코딩 벡터 계산
 ⇒ 토큰의 위치마다 동일한 임베딩 벡터 사용 X → 각 토큰의 위치 정보 모델이 학습 가능

7.1 위치 인코딩

- `PositionalEncoding` 클래스 : 입력 임베딩 차원 `d_model`, 최대 시퀀스 `max_len` 입력 받음
- 입력 시퀀스의 위치마다 sin, cos 함수로 위치 인코딩 계산
 - 수식

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

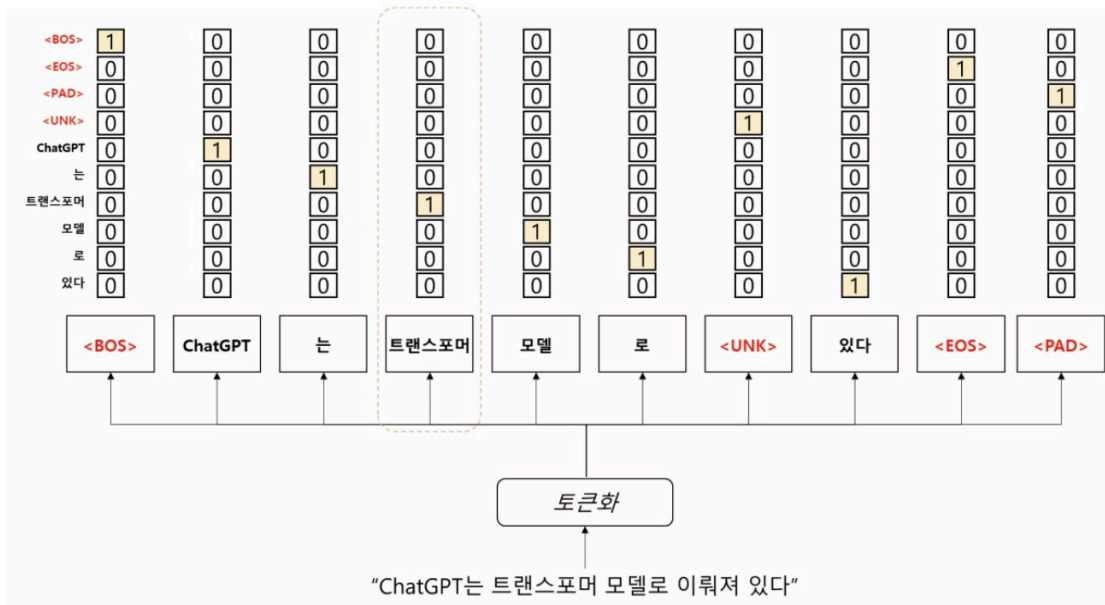
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

- `pos` : 입력 시퀀스에서 해당 단어의 위치
- `i` : 임베딩 벡터의 차원 인덱스
 - 짝수 → 첫번째 sin 함수 수식 적용
 - 홀수 → 두번째 cos 함수 수식 적용
- `1/10000^(2i/dmodel)` : 위치 인코딩에 사용, 각도 정보로 변환하기 위한 스케일링 인자로 각 위치에 대한 주기적인 신호 생성
- PE 변수의 텐서 차원 : [50, 1, 128] = [최대 시퀀스, 1, 입력 임베딩의 차원]
- 출력 결과 확인 시, 위치별 임베딩 차원이 주기적인 값으로 구성되는 것을 확인 가능
- 산출된 위치 인코딩은 입력 임베딩과 더해진 후 드롭아웃 적용됨
- `resister_buffer` 메서드 : 모델이 매개 변수 갱신하지 않도록 설정

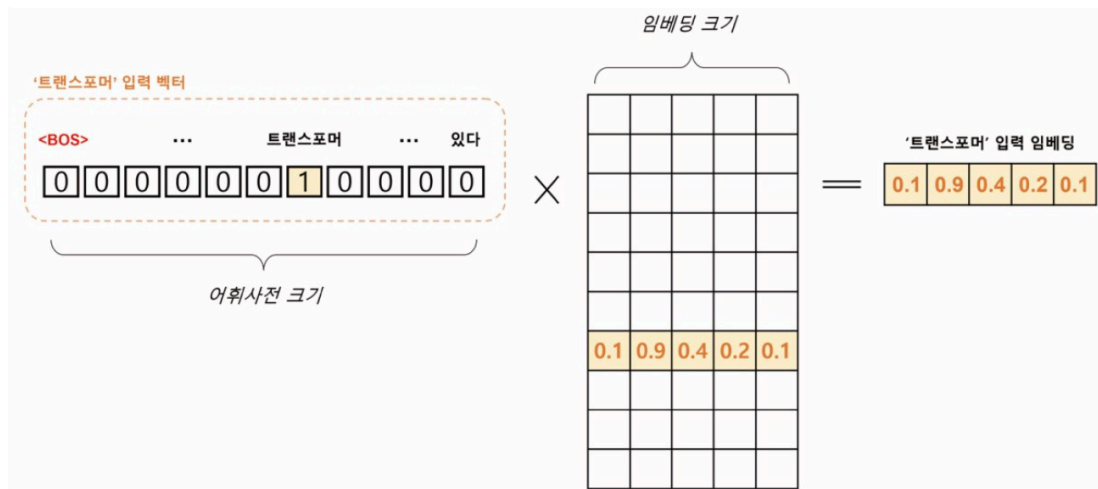
2. 특수 토큰

- 트랜스포머 : 단어 토큰 이외의 특수 토큰 활용하여 문장 표현

- 특수 토큰 : 입력 시퀀스의 시작과 끝을 나타내거나 마스킹 영역으로 사용
 - 모델이 입력 시퀀스의 시작과 끝을 인식할 수 있게 하며, 마스킹을 통해 일부 입력을 무시 가능
 - ex. 번역 모델 : 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조
- 특수 토큰을 활용한 문장 토큰 배열



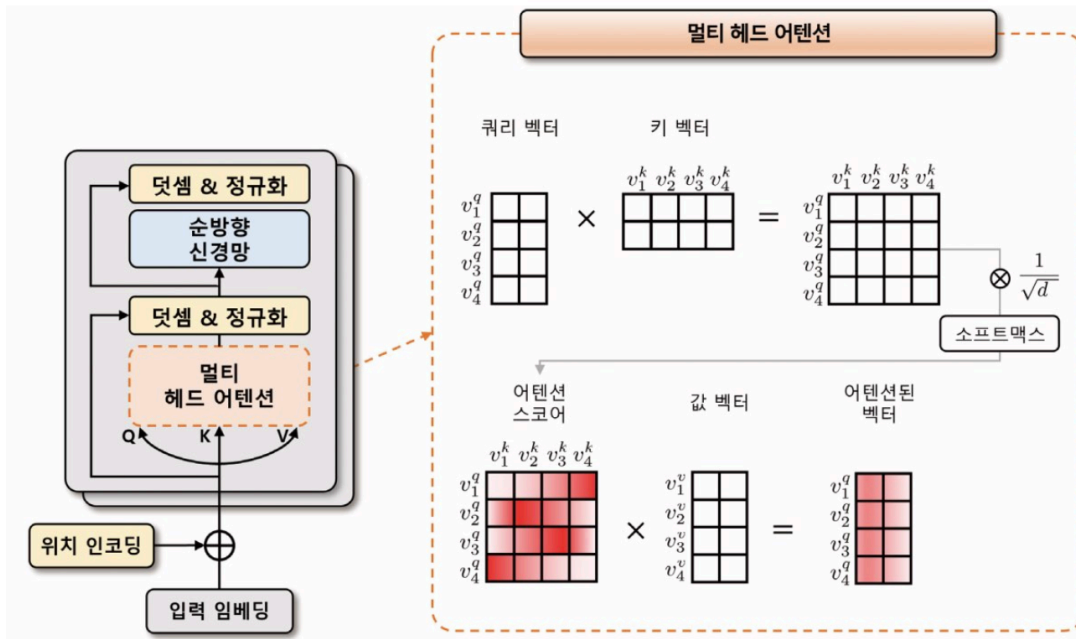
- **BOS** (Beginning of Sentence), **EOS** (End of Sentence), **UNK** (Unknown), **PAD** (Padding) 토큰 → 모두 특수 토큰, 자연어 처리에서 일반적으로 사용됨
 - **BOS** 토큰 : 문장의 시작
 - **EOS** 토큰 : 문장의 종료
 - 문장의 시작과 끝을 명확히 하기 위해 사용
 - **UNK** 토큰 : 어휘 사전에 없는 단어로, 모르는 단어 의미
 - 모델이 이전에 본 적이 없는 단어 처리할 때 사용
 - **PAD** 토큰 : 모든 문장을 일정한 길이로 맞추기 위해 사용
 - 짧은 문장의 빈 공간을 채울 때 사용
- 생성된 문장 토큰 배열 → 어휘 사전에 등장하는 위치에 원-핫인코딩으로 표현
- 입력 임베딩 산출 과정



- 입력 임베딩으로 변환되는 과정 : Word2Vec 방법과 동일
- 어휘 사전의 크기 = V , 입력 임베딩 차원 = d , '트랜스포머'의 원-핫벡터 크기 = $[1, V]$
 - 원-핫벡터는 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환
 - ⇒ 일반화 : N 개의 문장이 최대 S 개의 토큰 길이를 가질 때, $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 임베딩 텐서로 변환됨
 - 이 임베딩 텐서는 입력으로 사용, 트랜스 포머의 모든 계층에서 공유되는 텐서로 사용

3. 트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층 거쳐 연산 수행
- 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성, 입력 데이터에 대한 정보 추출 → 다음 계층으로 전달
- 인코더 계층에서 위치 정보 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해줌 → 산출된 인코더 계층의 출력은 디코더 계층으로 전달
- 트랜스포머 인코더 블록의 연산 과정



- 트랜스포머 인코더 : 위치 인코딩이 적용된 소스데이터의 입력 임베딩을 입력 받음
 - 멀티 헤드 어텐션 단계에서 입력 텐서 차원 = [N, S, d] → 입력 임베딩은 선형 변환 통해 3개의 임베딩 벡터 생성 = 쿼리(Q) , 키(K) , 값(V) 벡터
 - 쿼리 벡터 : 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터, 인코더의 각 시점 마다 생성, 현재 시점에서 질문이 되는 벡터를 의미하며 이 벡터를 기준으로 다른 시점의 정보 참조
 - 키 벡터 : 쿼리 벡터와 비교되는 대상, 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터가 됨, 키 벡터는 인코더의 각 시점에서 생성
 - 값 벡터 : 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할
- 쿼리, 키, 값 벡터는 초기에 비슷한 값을 가지지만, 모델 학습 통해 각 벡터는 의도한 의미의 값을 갖게 됨
- 쿼리 - 키 벡터의 연관성 : 내적 연산, [N , S , S] 어텐션 스코어 맵 사용

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 셀프 어텐션 과정에서는 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구함 → 이 스코어 값에 루트d (=벡터 차원의 제곱근)만큼 나눠 보정 → 이 보정 값은 벡터 차원이 커질 때 스코어 값이 같이 커지는 문제를 완화하기 위해 적용

- 보정된 어텐션 스코어를 소프트 맥스 함수를 이용하여 확률적으로 재표현
- 이를 값 벡터와 내적 → 셀프 어텐션된 벡터 생성
- ⇒ 이러한 과정 반복해 셀프 어텐션된 스코어 맵 생성
- 멀티 헤드 : 셀프 어텐션을 여러 번 수행해 여러개의 헤드 만들 → 각각의 헤드가 독립적으로 어텐션을 수행 → 그 결과를 합침
- 입력받는 $[N, S, d]$ 텐서에 k 개의 셀프 어텐션 벡터를 생성 : 헤드에 대한 차원 축을 생성 → $[N, S, d/k]$ 텐서 형태 구성 → 이 텐서는 k 개의 셀프 어텐션 된 $[N, S, d/k]$ 텐서 의미 → 생성된 k 개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 병합되어 $[N, S, d]$ 의 형태로 출력
- 덧셈 & 정규화는 멀티 헤드 어텐션을 통과하기 이전의 입력값 $[N, S, d]$ 텐서와 통과한 이후의 출력값 $[N, S, d]$ 텐서를 더함 → 학습시 발생하는 기울기 소실을 완화 + 이 값에 임베딩 차원축으로 정규화하는 계층 정규화를 적용
- 순방향 신경망은 두가지 방법 적용 가능 : 선형 임베딩과 ReLU로 이뤄진 인공 신경망이나 1차 원합성곱이 사용됨 → 이어서 다시 덧셈&정규화 과정이 수행됨
- 트랜스포머 인코더는 여러개의 트랜스포머 인코더 블록으로 구성됨
- 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록 통과 → 전차 입력 시퀀스 정보가 추상화
- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용되며, 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용 → 인코더-디코더가 서로 정보 공유

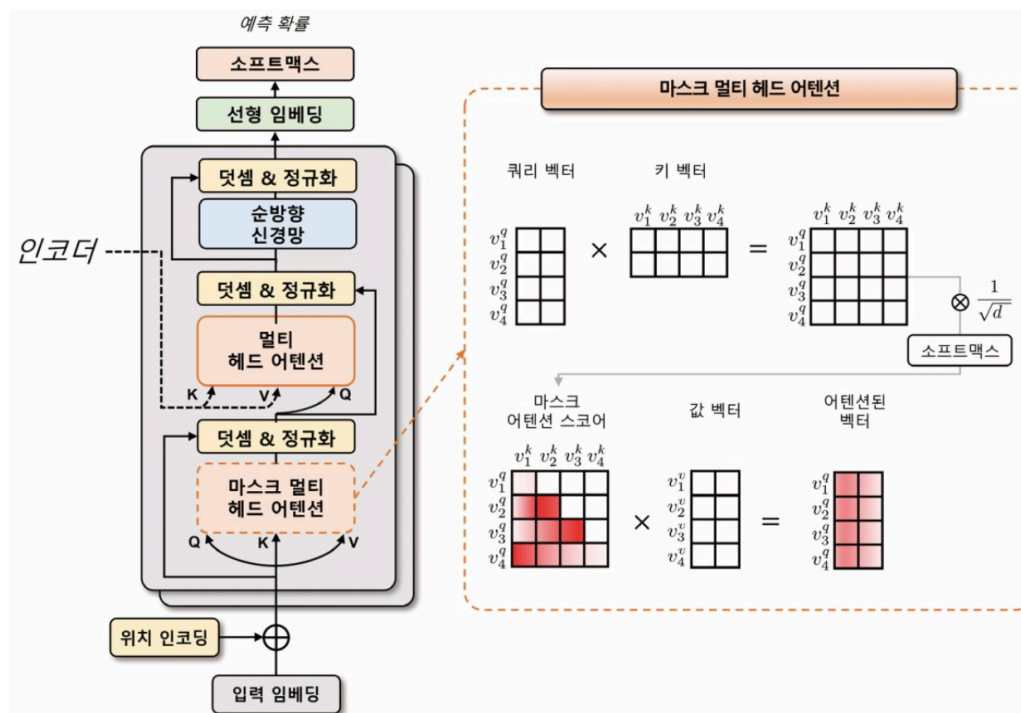
4. 트랜스포머 디코더

- 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력받음
 - 위치 인코딩: 입력 시퀀스 내에서 각 단어의 상대적인 위치 정보를 전달하는 기법 → 디코더의 입력 임베딩에 위치 정보 추가 → 디코더가 입력 시퀀스의 순서 정보 학습 가능
 - 트랜스포머 모델에서 인코더의 멀티 헤드 어텐션 모듈은 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체
 - 마스크 멀티 헤드 어텐션 모듈
 - 어텐션 스코어 맵 계산할 때 첫번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크 씌움
 - 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크 씌움
- ⇒ 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 됨, 인과성이 보장

- 마스크 영역에 수치적으로 굉장히 작은 값인 $-\text{inf}$ 마스크를 더해줌으로써 해당 영역의 어텐션 스코어 값이 0에 가까워짐
- 소프트 맥스 계산 시 해당 영역의 어텐션 가중치 = 0 $\rightarrow$ 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 됨

$\Rightarrow$ 마스크 멀티 헤드 어텐션 모듈은 인코더의 멀티 헤드 어텐션 모듈과 유사하지만, 인과성을 보장하면서 셀프 어텐션을 수행할 수 있게 됨

- 트랜스포머 디코더 블록의 연산과정



- 디코더의 멀티 헤드 어텐션에서는 타깃 데이터가 쿼리 벡터로 사용, 인코더의 소스 데이터가 키와 값 벡터로 사용 $\rightarrow$ 쿼리 벡터는 타깃 데이터의 위치 정보를 포함한 입력 임베딩과 위치 인코딩을 더한 벡터가 됨
- 쿼리벡터, 키벡터, 값벡터를 이용 $\rightarrow$ 어텐션 스코어 맵 계산 $\rightarrow$ 소프트 맥스 함수를 적용 $\rightarrow$ 어텐션 가중치 구함 $\rightarrow$ 어텐션 가중치와 값 벡터를 가중합 $\rightarrow$ 멀티 헤드 어텐션의 출력 벡터를 얻을 수 있음
- 디코더의 멀티 헤드 어텐션은 인코더의 멀티 헤드 어텐션과 거의 동일 but, 디코더에서는 셀프어텐션을 방지하기 위해 마스킹이 적용됨
- 트랜스포머의 디코더 : 인코더처럼 여러 개의 트랜스포머 디코더 블록으로 구성
 - 이전 트랜스포머 디코더 블록의 산출물 $\rightarrow$ 다음 트랜스포머 디코더 블록의 입력으로 전달 $\Rightarrow$ 디코더는 이전 시점에서의 정보를 현재 시점에서 활용할 수 있게

됨

- 마지막 디코더 블록의 출력텐서 $[N, S, d]$ 에 선형 변환 및 소프트맥스 함수 적용 → 각 타깃 시퀀스 위치 마다 예측 확률 계산 가능
- 디코더 : 타깃 데이터를 추론할 때 토큰 / 단어를 순차적으로 생성시키는 모델 → 입력토큰을 순차적으로 나타내는 방법 = 빈 값을 의미하는 PAD 토큰 사용
- 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

5. 모델 실습

- Multi30k 데이터세트 : 영어-독일어 병렬 말뭉치, 약 30,000개의 데이터 제공, 토치 데이터와 토치 텍스트 라이브러리로 해당 세트 쉽게 다운 가능
 - 토치 데이터 라이브러리 : 파이토치를 위한 텍스트 처리 라이브러리, 다양한 언어 모델링 작업에 대해 사전 처리 및 데이터 세트 관리를 단순화 하기 위한 다양한 도구와 기능 제공
 - 포트락커 라이브러리 : 파이썬에서 파일 락을 관리하기 위한 라이브러리, 파일 락을 사용해 여러 프로세스 간에 동시에 파일을 수정하거나 읽는 것을 방지
- Multi30k 데이터 세트를 다운로드 하고 압축 해제하는 과정에서 내부적으로 사용

7.2 데이터세트 다운로드 및 전처리

- `get_tokenizer` 함수 : 사용자가 지정한 토큰라이저를 가져오는 유틸리티 함수, `spacy` 라이브러리로 사전 학습된 모델을 가져옴 → 이 값을 `token_transform` 변수에 저장

- `vocab_transform` 변수 : 토큰을 인덱스로 변환시키는 함수 저장, Multi30k 데이터 세트를 활용해 (독일어, 영어)의 튜플형식으로 데이터를 불러옴
 - `build_vocab_from_iterator` 함수와 `generate_tokens` 함수로 언어별 어휘사전 생성
 - `build_vocab_from_iterator` 함수 : 생성된 토큰 이용해 단어 집합 생성
 - `min_freq` : 토큰화 된 단어들의 최소 빈도수 지정
 - `specials` (특수 토큰) : 트랜스포머에 활용하는 특수 토큰 지정, `Special_First` 매개변수가 참인 경우 특수 토큰을 단어 집합의 맨 앞에 추가
 - `set_default_index` 메서드 : 인덱스의 기본값을 설정 → 어휘 사전에 없는 토큰인 `<unk>`의 인덱스 할당

7.3 트랜스포머 모델 구성

- `Seq2SeqTransformer` 클래스 : `TokenEmbedding` 클래스로 소스 데이터와 입력 데이터를 입력 임베딩으로 변환하여 `src_tok_emb` 와 `tgt_tok_emb` 생성
 - 즉, 소스와 타겟 데이터의 어휘사전 크기를 입력받아 트랜스포머 임베딩크기로 변환
 - 이 입력 임베딩에 7.1 에서 정의한 `PositionalEncoding` 을 적용 → 트랜스포머 블록에 입력
- `Self.transformer` 트랜스포머 블록 : 파이토치에서 제공하는 트랜스포머 클래스를 적용
- 트랜스포머의 인코더와 디코더는 `encoder_layers` 변수의 값으로 구성
- 순방향 메서드 마지막에 적용되는 generator → 마지막 트랜스포머 디코더 블록에서 산출되는 벡터를 선형 변환 → 어휘 사전에 대한 로짓 생성
- 트랜스포머 클래스

```
transformer = torch.nn.Transformer(
    d_model=512,
    nhead=8,
    num_encoder_layers=6,
    num_decoder_layers=6,
    dim_feedforward=2048,
    dropout=0.1,
    activation=torch.nn.functional.relu,
    layer_norm_eps=1e-05,
)
```

- `d_model` : 임베딩 차원, 트랜스포머 모델의 입력과 출력 차원의 크기 정의, = 임베딩 차원 크기

- `nhead` : 멀티 헤드 어텐션의 헤드의 개수 정의, 헤드의 개수는 모델이 어텐션을 수행하는 방법 결정, 헤드의 개수↑ 모델 병렬처리 능력 ↑
but, 헤드의 개수 ↑ 모델 매개변수의 수 ↑
- `num_encoder_layers` / `num_decoder_layers` : 인코더와 디코더의 계층수를의미, 모델의 복잡도와 성능에 영향을 미침, 계층 개수 ↑ 더 복잡한 문제를 해결가능 but, 너무 많을 경우 과대적합 가능
- `din_feedforward` : 순방향 신경망 크기, 순방향 신경망의 은닉층 크기 정의
→ 트랜스포머 계층의 각 입력 위치에 독립적으로 적용됨, 인코더/디코더 계층과 마찬가지로 모델의 복잡도와 성능에 영향 미침
- `dropout` : 인코더와 디코더 계층에 적용되는 드롭아웃 비율 적용
- `activation` : 활성화 함수, 순방향 신경망에 적용되는 활성화함수 의미, 활성화 함수는 파이토치 함수 형태로 입력 가능
- `layer_norm_eps` : 계층 정규화 입실론, 계층 정규화 수행 시 분모에 더해지는 입실론 값 정의
- 트랜스포머 순방향 메서드

```
output = transformer.forward(
    src,
    tgt,
    src_mask=None,
    tgt_mask=None,
    memory_mask=None,
    src_key_padding_mask=None,
    tgt_key_padding_mask=None,
    memory_key_padding_mask=None,
)
```

- `src` / `tgt` : 소스와 타겟, 인코더와 디코더에 대한 시퀀스, [소스(타겟), 시퀀스 길이, 배치 크기, 임베딩 차원] 형태의 데이터를 입력받음
- `src_mask` / `tgt_mask` : 소스와 타겟 시퀀스의 마스크로 [소스(타겟)시퀀스 길이, 시퀀스 길이] 형태의 데이터 입력 받음
 - if 마스크 값 = 0 : 해당 위치에서는 모든 입력 단어가 동일한 가중치 → 어텐션이 수행됨

- if 마스크 값 = 1 : 모든 입력 단어의 가중치가 0으로 설정 → 어텐션 연산 수행 X
- if 마스크 값 = -inf : 해당 위치에서는 어텐션 연산 결과에 0으로 가중치 부여 → 마스킹된 위치의 정보를 모델이 무시하게 만듦
- if 마스크 값 = +inf : 모든 입력 단어에 무한대의 가중치가 부여 → 어텐션 연산 결과가 해당 위치에 대한 정보만으로구성
→ 일반적으로 +inf는 적용하지 X, 어떤 특정 단어나 위치에 대해 모델이 특별한 관심을 가지도록 할 때만 사용
- `memory_mask` : 메모리 마스크, 인코더 출력의 마스크로 [타겟 시퀀스길이, 소스 시퀀스 길이]의 형태, 메모리 마스크의 값이 0인 위치에서는 어텐션 연산 수행 x
- `key_padding_mask` : 소스,타겟, 메모리 시퀀스에 대한 패딩 마스크 의미, [배치크기, 소스(타겟)시퀀스길이] 형태의 데이터를 입력 받음
→ 메모리 키 패딩 마스크는 소스 키 패딩 마스크와 동일한 형태의 데이터를 입력 받음
- 키 패딩 마스크 : 입력 시퀀스에서 패딩 토큰이 위치한 부분을 가리키는 이진 마스크, 패딩 토큰이 실제 의미 가지지 않는 것으로 간주 → 해당 위치의 어텐션 연산 결과에 대한 가중치 = 0
- 순방향 메서드 : 인스턴스의 설정과 입력 시퀀스를 통해 타겟 시퀀스의 임베딩 텐서를반환 = [타겟 시퀀스길이, 배치 크기, 임베딩 차원] 반환

7.4 트랜스포머 모델 구조

- `Seq2SeqTransformer` 클래스 : 입력 임베딩 `src_tok_emb`, `tgt_tok_emb`, 위치인코딩 `positional_encoding`, 트랜스포머 블록 `transformer`, 로짓 생성 `generator` 로 구성
- 패딩 토큰은 모델 학습에 사용 X → 해당 토큰에 대한 레이블 무시, 모델이 해당 클래스를 학습하지 않게 함 ⇒ 이 클래스에 대한 손실은 계산 X 모델이 해당 클래스를 예측하더라도올바른 예측으로 간주 X

7.5 배치 데이터 생성

- `sequential_transforms` 함수 : 여러 개의 전처리 함수를 인자로 받아 이를 차례로 적용하는 함수를 반환하는 함수
 - `token_transform` : 문장 토큰화
 - `vocab_transform` : 각 토큰 인덱스화
 - `input_transform` 함수 : 인덱스화된 코트에 문장의 시작 BOS_IDX(2)와 끝 EOS_IDX(3) 을 알리는 특수 토큰 할당

- `collator` 함수 : `rstrip("\n")` 함수로 문자열의 끝에 있는 (\n)를 제거하고, `text_transform` 변수에 저장된 `sequential_transforms` 함수 적용
- `pad_sequence` 함수 : 소스와 타겟 시퀀스를 패딩 ,패딩시퀀스함수는 동일한 길이를 가지도록 시퀀스의 뒤쪽에 PAD_IDX(1)로 채워진 패딩 토큰을 추가

7.6 어텐션 마스크 생성

- 트랜스포머 모델에서 사용되는 마스크를 생성하는 함수와 두 시퀀스의 패딩 마스크를 생성 하는 함수로 구성
- `generate_square_subsequent_mask` : 마스크를 생성하는 함수, 입력으로 정수 s를받아 sxs 크기의 마스크를 생성, 함수를 적용하여 상삼각행렬을 생성, 마지막으로 `transpose` 함수를 사용하여 행렬을 전치
- `create_mask` : 패딩 마스크를 생성하는 함수, 시퀀스를 입력 받아 길이를 계산하고마스크 생성 함수로 타겟 시퀀스의 마스크를 생성
- 소스 마스크 : 소스 시퀀스 길이의 크기로 채워진 행렬을 생성, 패딩마스크는 각 시퀀스에 대해 패딩 토큰의 위치를 찾고 `transpose` 함수로 전치
- 패딩마스크를 생성하기 전에, 타겟 데이터의 입력값(target_input)과 출력값(target_out)은 토큰 순서를 한 칸 시프트(Shift)하여 이전 토큰들이 주어졌을 때 다음 토큰을 예측
- 패딩 마스크 생성 함수로 소스 시퀀스 입력값 / 타겟 시퀀스 입력값을 전달해 4개의 텐서를 생성시킴
- `source_mask` : 셀프 어텐션 과정에서 참조되는 소스 데이터의 시퀀스 범위
 - False인 위치 : 셀프 어텐션에 참조되는 토큰
 - True인 위치 : 텐션에서 제외되는 토큰
- `target_mask` : [쿼리 시퀀스 길이, 키 시퀀스 길이]의 형태로 구성되며, 1번 째 쿼리 벡터는 i+1 이상의 키 벡터에 대해 어텐션 연산을 수행할 수 없게 됨
- 모델이 현재 예측하고자 하는 위치 이전의 토큰들만 참고 하게 제한함으로써, 모델이 미래시점의 정보를 사용하지 않게 해 현재 시점에 영향을 미치지 않게 함
- `source_padding_mask` , `target_padding_mask` : 소스 배치 데이터에서 텍스트 토큰이 존재하는지 여부를 나타내는 값
 - False인 경우 : 해당 토큰 인덱스가 존재
 - True인 경우 : 해당 토큰 인덱스가 패딩 토큰으로 채워져 있음

7.7 모델 학습 및 평가

- `run` 함수 : 모델 학습 및 평가를 위한 함수로 소스와 타겟 데이터를 입력받아 `collator` 로 문장들을 토큰화하고 인덱스로 변환
- `create_mask` 함수 : 트랜스포머 모델에 필요한 입력패딩 마스크 (`src_padding_mask`, `tgt_padding_mask`)와 어텐션마스크(`src_mask`, `tgt_mask`)를 생성 → 이 결과값들은 타겟 시퀀스의 1번째 까지 토큰이 주어졌을 때 1+1번째 토큰을 예측하는 데 활용

7.8 트랜스포머 모델 번역 결과

- 모델 번역 방식은 그리드 디코딩(Greedydecoding)방식으로 번역 결과 출력
- 그리드 디코딩 : 디코더 네트워크가 생성한 확률 분포에서 가장 높은 확률을 가지는 단어를 선택하는 방법, 현재 시점에서 가장 확률이 높은 단어를 선택하여 디코딩 진행
- 모델 추론 방식: 디코더에 참조되는 마지막 인코더 트랜스포머 블록의 벡터 `memory` , 타겟데이터의 입력 텐서 `ys` , 타겟 마스크 `target_mask` 사용
- `memory` 텐서 : 생성하려면 소스 문장을 토큰 인덱스로 표현한 `source_tensor` 를 생성하고, `source_mask` 소스 문장 모든 토큰이 어텐션 될 수 있게 0 값으로 설정
- `model.encode` 메서드 : `source_tensor` 와 `source_mask`를 입력으로 넣어 소스문장에 대한 인코딩을 수행 → 마지막 인코더 트랜스포머 블록의 벡터 추출
 - `model.encode` 메서드에서 생성된 out : [토큰 개수, 배치 크기, 확률]의 형태
 - 이 값을 `transpose` 함수를 이용하여 [배치크기,토큰개수, 확률] 형태로 변환 → 텐서를 슬라이싱해 [배치 크기, 확률] 형태로 만듦 → 어휘 사전에서 가장 확률이 높은 토큰 인덱스를 찾음
 - ⇒ 이 과정을 `max_len` 이전이거나 `EOS_IDX` 를 예측할 때까지 반복, 최종적으로 예측된 토큰 시퀀스 반환
 - `max_len` = `num_tokens`+5로 구성
 - `num_tokens` : 소스 문장의 토큰 개수
 - 일반적으로 생성된 문장의 길이가 소스 문장 길이 보다 약간 더 길어지는 경우가 많기 때문에 5를 더함
 - `greedy_decode` 함수 : 현재까지 예측된 토큰들을 이용해 다음 토큰을 예측하는 데, 이 때 가장 높은 확률을 가지는 단어를 선택하여 다음 토큰으로 예측
 - 마지막으로 예측된 토큰 인덱스를 어휘사전의 `lookup_tokens` 함수를 통해 텍스트로 변환 → 후처리 : <bos>와 <eos>토큰은 슬라이싱([1:- 1])으로 제거

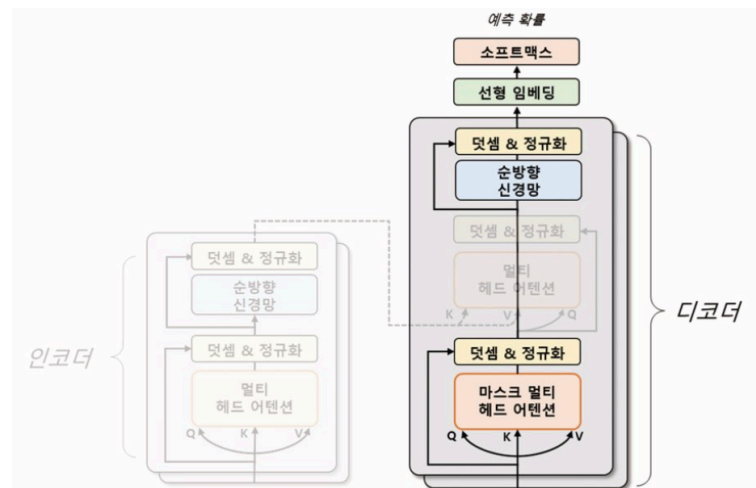
2 GPT

- 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델로 가장 널리 알려진 언어모델

- 모델은 ELMo와 같이 대규모 말뭉치로 사전 학습된 모델로 다양한 다운스트림 작업에서 우수한 성능
- 트랜스포머의 디코더를 여러층 쌓아만든 언어 모델
- 트랜스포머의 인코더는 입력 문장의 특징을 추출하는 데 초점을 두고 있다면, 디코더는 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점
→ 이러한 특성 때문에 디코더를 쌓아 모델을 구성하는 것이 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능 발휘
- GPT 모델은 대규모 텍스트 데이터 세트로 사전 학습해 초기화되며, 특정 자연어 처리 작업에 맞게 미세 조정해 사용
- 일반적으로 자연어 생성, 기계 번역, 질의 응답, 요약 등 다양한 자연어 처리 작업에 활용되며, 예측 성능이 뛰어나고 적은 데이터로도 높은 성능을 보임
- GPT모델 시리즈
 - 모델들은 거의 동일한 트랜스포머 구조를 사용하며, 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터를 활용해 학습됨
 - GPT- 1 : 첫번째 GPT모델로, 약 1억2천만 개의 매개변수가 존재
 - GPT- 2 : 두번째 발표된 모델로, GPT-1의 성능을 크게 능가하며 약 15억 개의 매개변수가 존재
 - GPT- 3 : GPT-2보다 훨씬 더 큰 1, 750억 개의 매개변수 갖는 초거대모델로서 매우 뛰어난 문장 생성 성능을 보여줌.
but, GPT- 3는 막대한 양의 매개변수를 가진모델이기 때문에 문장을 생성하는 데 많은 자원을 필요로 함
→ 많은 양의 매개변수를 줄이기 위해 Open AI는 RLHF(Reinforcement Learning from Human Feedback) 방법을 도입해 GPT- 3. 5 (InstructGPT)모델 학습
 - RLHF : 인간의 피드백을 이용하는 강화학습방법, 인간이 모델이 생성한 결과물을 평가하고, 모델이 좋은 결과물을 생성하면 보상을 주어서 모델을 학습
→ 인간의 지도를 통해 모델 학습시킨 방법은 기존 학습 데이터만 사용하는 것보다 더욱 정확하고 효율적인 모델 학습이 가능해 약 60억 개의 가중치로도 뛰어난성능을 낼 수 있게 됨
 - 22년 10월에는 대화형 질의 응답 시스템으로 사용할 수 있는 ChatGPT 서비스가 공개됨, GPT- 3.5를 기반, 사용자와의 대화를 통해 지속적으로 학습해 발전
 - 23년 3월에는 이미지 데이터를 함께 학습하는 GPT- 4모델이 발표

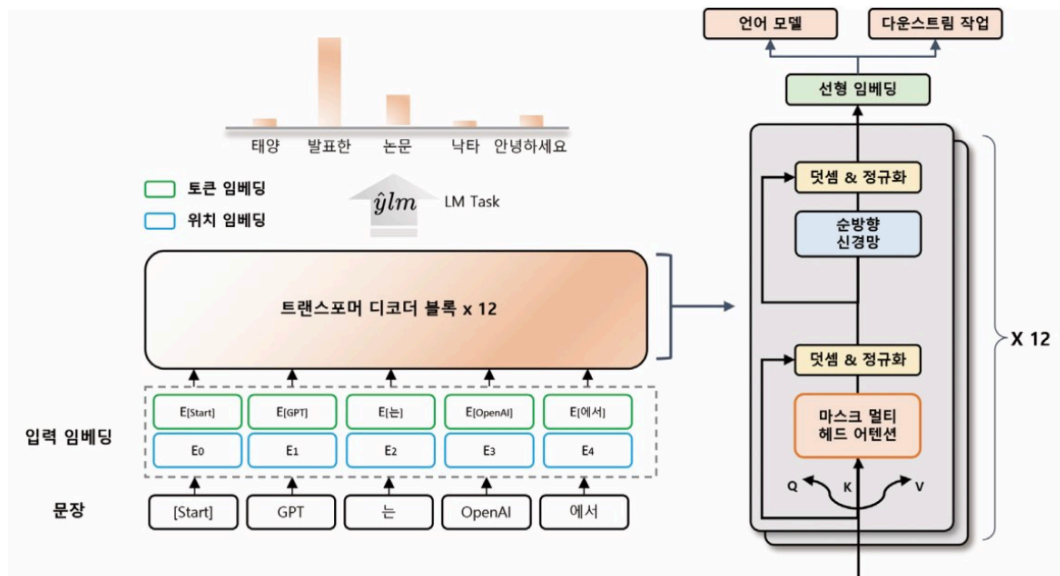
1. GPT-1

- 트랜스포머 구조를 바탕으로 한 단방향 언어 모델
 - 단방향 언어 모델 : 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측하는 모델
 - = 이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작
 - 이러한 방식은 모델의 계산량 ↓ ⇒ 학습속도 ↑ , 모델 크기 ↓ 장점
- 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조 활용
 - : 트랜스포머 구조 중 디코더 부분만 사용, 인코더- 디코더 간 어텐션 연산 제외
 - 모델의 매개변수 ↓ 성능 ↑
- 트랜스포머와 GPT-1 모델 비교

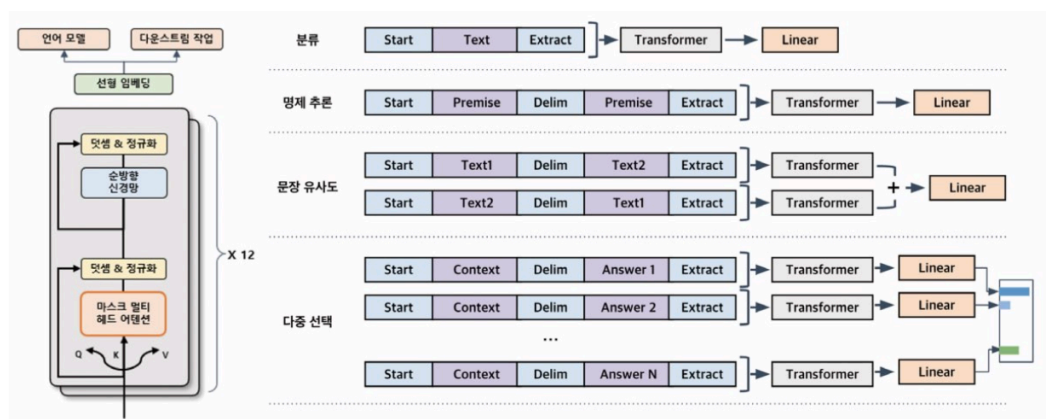


- GPT-1 모델
 - 인코더 사용 X, 디코더 부분만 사용하여 모델 구성
 - 디코더의 두번째 다중 헤드 어텐션 사용 X
 - 12개의 헤드를 가진 트랜스포머의 디코더를 12층 쌓아 모델을 구성 → 약 1억2천만 개의 매개변수로 학습
 - 약 4.5GB의 B00kCorpus 데이터셋을 사용해 언어 모델 사전학습
 - 이때 입력 문장을 임의의 위치까지 보여주고, 뒤에 이어지는 문장을 모델이 자동으로 예측하게 함
 - ⇒ 별도의 레이블이 필요하지 않음 → 비지도 학습으로 학습
 - = 컴퓨팅 자원만 충분하다면 제약없이 학습할 수 있다는 장점

- GPT-1의 사전 학습 작업 구조



- 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어모델을 통해 사전 학습
→ 입력 문장을 일정한 길이의 블록으로 나누어 처리, 각 블록에서는 블록 내의 첫번째 토큰 부터 순서대로 다음 토큰 예측
⇒ 모델이 장기적인 문맥을 이해할 수 있게 함
- 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산
 - 토큰 임베딩 : 각 토큰을 고정된 차원의 벡터 매핑
 - 위치 임베딩 : 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터 매핑
- GPT-1의 다운스트림 작업 구조

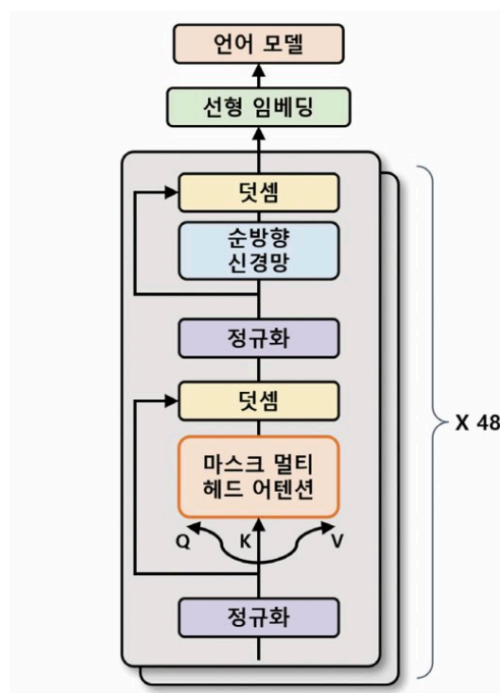


- 미세 조정을 위한 다운 스트림 작업에서도 GPT-1은 각 작업에 따라 입출력 구조를 바꾸지 않고 언어모델을 보조학습해 사용할 수 있음

- 보조 학습: 모델이 주어진 주요 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시에 수행하는 것을 의미, 보조 작업에서 학습한 정보를 주요 작업에 전달하여 성능을 향상시킬 수 있음
- 다운스트림 작업에서는 원하는 목표에 맞게 모델을 세밀하게 조정 필요 → 일반적인 언어모델 학습 시 사용되는 손실 함수 외에 다운스트림 작업에서 필요한 추가 손실 함수가 필요
⇒ 두 개의 손실함수 사용해 최적화 → 보조 학습을 통해 언어모델 개선 + 다운스트림 작업의 목표를 달성하기 위해 필요한 정보 학습 가능
- 특수 토큰 사용 : 문장의 시작을 의미하는 <start> 토큰, 두 문장의 경계를 의미하는 <delim> 토큰, 문장의 마지막을 의미하는 <extract>토큰 사용

2. GPT-2

- 1024의 길이까지 입력 처리 가능 (↔ GPT-1 : 최대 512)
- 48 개의 디코더 계층 (↔ GPT-1 : 12개)
- 15억 개의 매개변수 (↔ GPT-1 : 1,200만 개)
→ GPT-1 보다 더욱 복잡한 패턴 학습 가능
- GPT-2 모델 구조



- GPT-2는 미국의 웹사이트에서 스크래핑한 약 8백만개의 문서를 사용해 약 40GB 데이터로 사전학습 수행 (> GPT-1)

→ 더욱 자연스러운 문장 생성 능력, 더 다양한 분야 활용

- 제로-샷 학습 가능 : 별도의 미세조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습과정에서 특정한 형식의 데이터 입력

3. GPT-3

- GPT- 2와 거의 동일한 모델구조 but, 몇몇 모델 매개변수의 변화 → GPT-2보다 약 116배 많은 매개변수
- 96개의 헤드를 가진 멀티 헤드 어텐션, 96개의 트랜스포머 디코더 층 사용
→ 멀티 헤드 어텐션 과정에서 연산량을 줄이기 위해 희소어텐션과 일반어텐션 섞어 사용
- 토큰 임베딩의 크기 1,600 → 12,888 대폭증가
- 토큰 2,048개 까지 입력 가능 (↔ GPT-2 : 최대 1,024개)
- 웹 크롤링, 위키백과, 서적 등에서 수집한 약 45TB에 달하는 방대한 양의 데이터 세트를 이용하여 모델을 학습
→ 이 모델은 다운스트림 작업으로 학습하지 않았지만, 질의 응답, 번역, 요약, 문서 생성 등 다양한 다운스트림 작업에서도 높은 성능
- 프롬프트(prompt)형식으로 작업 수행

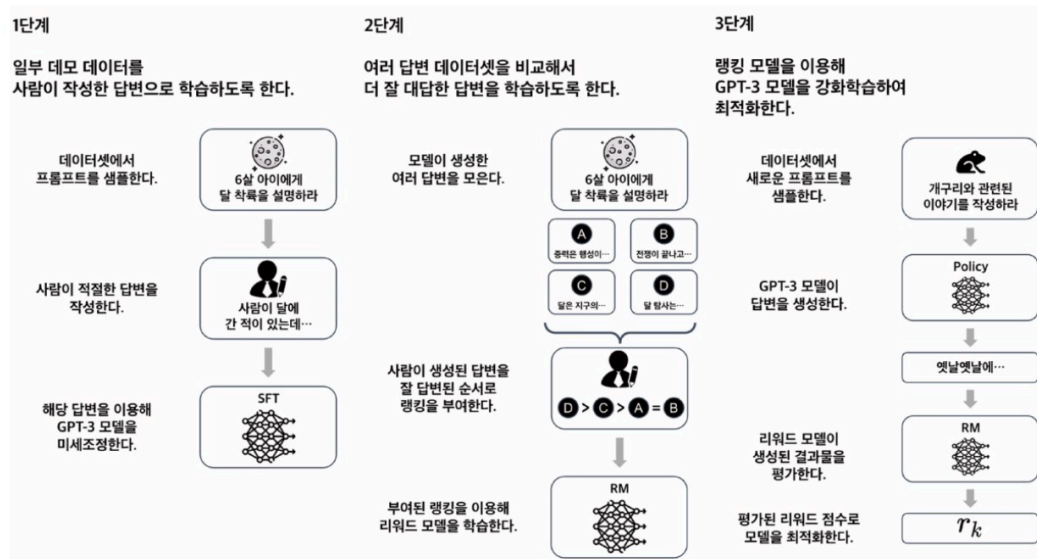
질의 응답 입력 프롬프트 문서 : 생성적 사전학습 트랜스포머(Generative Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다. 질문 : GPT-3를 만든 곳은 어디인가? 답변 : OpenAI 출력값	자연어 추론 입력 프롬프트 자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다. 질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님 답변 : 둘 다 아님 출력값
수학 문제 풀이 입력 프롬프트 질문 : $(2 * 4) * 6$은 무엇인가? 답변 : 48 출력값	일반 상식 질의 응답 입력 프롬프트 질문 : 물은 섭씨 몇 도에서 어는가? 답변 : 0도 출력값

- 새로운 도메인에서의 작업에 대해서도 높은 일반화 능력 → 질의응답, 수학 문제 풀이, 자연어 추론 등과 같은 기존 자연어 처리 분야에서는 이전의 기술적 한계를 극복
- 큰 모델 → 매우 느리고 비용이 많이 듦 → 일반 사용자나 소규모 기업에서 활용하기 어려움 + 사전 학습된 데이터에 기반하여 작동하기 때문에 데이터가 편향돼 있거나 정확하지 않을 경우 오류 발생

- 자연어 처리 작업에 대한 일반화 능력이 높지만, 사람과 비교할 때 인간적인 이해력과 상식적인 추론능력이 부족, 거짓, 편견, 혐오 등이 반영될 가능성

4. GPT-3.5

- GPT-3의 모델 구조를 그대로 따르면서도, 새로운 학습방법인 RLHP를 도입해 모델의 매개변수 수를 줄이고 모델의 자연스러움을 높인 것이 특징
- RLHF GPT-3 학습방법



- 데이터 세트에서 임의의 프롬프트를 가져옴 → 이 프롬프트에 대해 사람이 직접 적절한 답을 작성 → 작성된 답변은 모델이 생성한 문장과 함께 평가해 모델이 생성한 문장이 자연스러우면 보상 / 그렇지 않으면 보상 X
→ 이러한 보상을 바탕으로 강화 학습을 통해 모델 학습
= 지도 미세 조정 (Supervised Fine-Tuning, SFT)
- 사람 답변 생성에 한계 존재 → GPT 모델이 얼마나 잘 답변 했는지 평가하는 보상 모델을 학습
- GPT 모델은 시드(seed)에 따라 다른 답변을 생성하므로 내용이 서로 다른 여러 개의 답변이 생성
- 사람이 부여한 랭킹은 생성된 답변의 우수성 평가한 결과이며, 이를 보상 모델의 입력으로 사용 → 이렇게 생성된 답변과 랭킹 정보를 활용하여 학습 → 모델은 더 자연스러운 문장 생성
- 환각(Hallucination) 현상 : 모델이 인간과 같은 추론과 판단 능력을 갖추지 못하고 부적절 하거나 오류가 많은 답변을 제시하는 현상

5. GPT-4

- 텍스트 데이터 뿐만 아니라 이미지 데이터도 인식 가능한 멀티모달 모델
- 최대 32,768개의 토큰 입력 가능 ($\leftrightarrow$ GPT-3.5 : 4,096개)
- 25,000개의 단어 처리 가능 ($\leftrightarrow$ GPT-3.5 : 3,000개)
- 대규모 다중 작업 언어 이해 85% 성능 ($\leftrightarrow$ GPT-3.5 : 77%)

6. 모델 실습

7.9 문장 생성을 위한 GPT-2 모델의 구조

- 간소화된 GPT-2 모델의 구조 : 단어 토큰 임베딩 `wte`, 단어 위치 임베딩 `wpe`, 드롭 아웃 `drop`, 트랜스포머 디코더 계층 `h`, 선형 임베딩 및 언어모델 `lm_head`
- `pipeline` 함수: 문장분류, 문장생성, 토큰분류 등 다양한 작업에 대한 전처리, 모델아키텍처, 후처리를 각각 처리

7.10 GPT-2를 이용한 문장 생성

- 파이프라인 클래스 : 입력된 작업(task)에 모델(model)로 적합한 파이프라인 구축, 작업매개 변수는 수행하려는 작업 의미
- 파이프라인 함수 : 설정한 작업을 수행하는 파이프라인 클래스의 인스턴스 반환
- `text_inputs` : 입력 텍스트, 생성 하려는 문장의 입력 문맥을 전달,
- `max_length` : 최대 길이, 생성될 문장의 최대토큰수를 제한한다.
- `num_return_sequences` : 반환 시퀀스 개수, 생성할 텍스트 시퀀스의 개수
- `pad_token_id` : 패딩 토큰 ID, 모델의 자유생성 여부 설정
→ 해당 매개변수를 사용하면 모델이 입력된 문장의 문맥을 기반으로 자유롭게 다음 단어나 문장을 생성

7.11 CoLA 데이터셋 불러오기

- CoLA 데이터셋 : train, dev, test 데이터셋 구성
- `collator` 함수 : 배치를 토큰나이저로 토큰화하고, 패딩 `pading`, 절사 `truncation`, 반환 형식 설정 `return_tensors` 작업을 수행
- 패딩에 인자를 `longest`로 설정하면 가장 긴 시퀀스에 대해 패딩을 적용, 절사에 인자를 `True`로 설정하면 입력시퀀스 길이가 최대 길이를 초과하는 경우 해당 시퀀스를 자름
- 반환 형식 설정에 `pt`를 입력 → 파이토치 텐서 형태로 결과 반환
- GPT 모델 : 어텐션 마스크를 이용해 입력 문장에서 패딩 부분을 무시하고 실제 단어에 대해 처리

- 토큰 ID : 토크나이저가 각 토큰에 대해 부여한 숫자ID를 담고 있음
- 어텐션마스크: 입력 문장의 실제 단어에 대응하는 부분은 1로, 패딩에 대응하는 부분은 0으로 채움

7.12 GPT-2 모델 설정

- `GPT2ForSequenceClassification` 클래스: GPT-2 모델을 기반으로 하는 시퀀스 분류 모델, 기본 GPT-2 모델과 유사하게 작동하지만, 최종 출력 계층이 분류를 위해 미세조정됨
- GPT-2는 사전 학습 시 토큰의 패딩 기법 사용 X → GPT-2의 토크나이저에는 패딩 토큰이 포함돼 있지 않음
- 문장 분류 모델을 학습하기 위해서는 모델이 고정된 길이의 입력을 필요 → 문장의 끝을 나타내는 eos 토큰을 사용해 패딩 토큰으로 대체

7.13 GPT-2 모델 학습 및 검증

7.14 모델 평가